

Persistencia en archivos y subida de fotos

Que tus datos no se pierdan al reiniciar el servidor

EL PROBLEMA QUE VAMOS A RESOLVER

Hasta ahora las personas se guardan en un arreglo dentro del código. Funciona... hasta que apagas el servidor. Cierras la terminal, vuelves al rato, y todas las personas desaparecieron. En este paso lo arreglamos.

QUE VAMOS A HACER

Parte A. Guardar las personas en un archivo **personas.json**. Cuando crees una persona se escribe al archivo. Cuando arranque el servidor, lo lee.

Parte B. Agregar un campo "foto de perfil" al formulario. El archivo se sube al servidor con **multer** y la ruta queda guardada con los demás datos.

DE DONDE PARTIMOS

Del código que ya tenías: el servidor con formulario de personas (nombres, apellidos, edad, dirección, password) que las guarda en un arreglo en memoria. Esto es lo que vamos a mejorar.

PARTE A - Persistencia en archivo JSON

Antes de programar: que es persistencia

Persistir significa que los datos sobrevivan despues de que el programa termine. Hay varias formas de hacerlo:

Forma	Cuando se usa
Archivo JSON (lo que haremos)	Prototipos, datos pequenos, configuracion. Facil de entender y debuggear.
Base de datos SQL	Aplicaciones reales con muchos datos relacionados (MySQL, PostgreSQL).
Base de datos NoSQL	Datos sin estructura fija o con mucho volumen (MongoDB, Redis).

La idea (en 3 frases)

El servidor sigue trabajando con un **arreglo en memoria** (es lo rapido). La diferencia es que, al iniciar, ese arreglo se carga desde el archivo JSON. Y cada vez que cambia, se vuelve a escribir al archivo.

Las herramientas: fs y path (vienen con Node)

- **fs** (*file system*): leer y escribir archivos. No hay que instalar nada, es parte de Node.
- **path**: armar rutas de archivos compatibles con cualquier sistema operativo (Windows usa \, Mac y Linux usan /).
- **JSON.stringify** y **JSON.parse**: convertir entre objetos JavaScript y texto JSON.

A.1 Crear la carpeta data/

En la raíz de tu proyecto, crea una carpeta llamada **data**. Aquí va a vivir el archivo donde guardaremos las personas. Mantenerlo aparte (no mezclado con tu código) es buena práctica.

```
mi-proyecto/  
+-- index.js  
+-- package.json  
+-- data/          # <- carpeta nueva  
|   +-- (vacía)    # se llenará cuando crees personas  
+-- public/  
+-- views/
```

No crees el archivo personas.json a mano

El código lo va a crear solo la primera vez que guardes una persona. Solo necesitas la carpeta **data/**.

A.2 Importar fs y path en index.js

Arriba de tu **index.js**, junto a los otros **require**, agrega estos dos:

```
const express = require("express");  
const { engine } = require("express-handlebars");  
const fs = require("fs");          // nuevo  
const path = require("path");      // nuevo
```

No necesitas hacer **npm install**: ambos son módulos nativos de Node, ya están disponibles.

A.3 Definir la ruta del archivo

Después de la configuración de Express, define una constante con la ruta completa al archivo:

```
// Ruta del archivo donde guardamos los datos  
const ARCHIVO_PERSONAS = path.join(__dirname, "data", "personas.json");
```

Que es `__dirname` y por que `path.join`

- **__dirname** es una variable que Node nos da gratis: contiene la ruta de la carpeta donde está el archivo que se está ejecutando.
- **path.join** arma rutas que funcionan en Windows y en Mac/Linux. Si pusieras **"data/personas.json"** a mano podría fallar en Windows.
- El resultado será algo así: **C:\Users\Angell\mi-proyecto\data\personas.json** (en Windows) o **/Users/angel/mi-proyecto/data/personas.json** (en Mac).

A.4 Funcion para cargar las personas desde el archivo

Esta funcion lee el archivo y devuelve el arreglo. Si el archivo no existe todavia (es la primera vez que corres el servidor), devuelve un arreglo vacio.

```
function cargarPersonas() {
  try {
    // Si el archivo no existe, devolvemos arreglo vacio
    if (!fs.existsSync(ARCHIVO_PERSONAS)) {
      return [];
    }
    const contenido = fs.readFileSync(ARCHIVO_PERSONAS, "utf-8");
    return JSON.parse(contenido);
  } catch (error) {
    console.error("Error leyendo personas.json:", error.message);
    return [];
  }
}
```

Que hace cada parte

- **fs.existsSync(...)** revisa si el archivo existe. Si no, devolvemos arreglo vacio (no es un error, es el primer arranque).
- **fs.readFileSync(...)** lee el archivo y devuelve su contenido como texto. El "utf-8" le dice como interpretar los caracteres.
- **JSON.parse(...)** convierte ese texto en un objeto/arreglo de JavaScript.
- El **try/catch** protege contra archivos corruptos (alguien edito el JSON a mano y rompio la sintaxis).

A.5 Funcion para guardar las personas en el archivo

Esta funcion toma el arreglo y lo escribe al archivo en formato JSON:

```
function guardarPersonas(personas) {
  fs.writeFileSync(
    ARCHIVO_PERSONAS,
    JSON.stringify(personas, null, 2),
    "utf-8"
  );
}
```

El truco del null, 2

Los dos ultimos argumentos de **JSON.stringify** hacen que el archivo quede indentado y legible (con 2 espacios). Sin eso te queda todo en una sola linea gigante y dificil de leer.

A.6 Cambiar el arreglo vacio por una llamada a cargarPersonas

En tu codigo original tenias:

```
const personas = [];
```

Cambialo por:

```
let personas = cargarPersonas(); // carga al iniciar
```

Fijate en dos cosas: cambiamos **const** por **let** (vamos a reasignar el arreglo si recargamos) y llamamos a **cargarPersonas()** para que el arreglo arranque con lo que haya en el archivo.

A.7 Llamar a guardarPersonas despues del push

En la ruta **POST /personas**, despues del **personas.push(...)**, llama a **guardarPersonas**. Asi cada cambio queda persistido inmediatamente:

```
app.post("/personas", (req, res) => {
  const { nombres, apellidos, edad, direccion, password } = req.body;

  personas.push({
    nombres,
    apellidos,
    edad,
    direccion,
    password
  });

  guardarPersonas(personas); // <- linea nueva

  res.redirect("/personas");
});
```

A.8 Probar que funciona

Levanta el servidor con **node index.js** y haz lo siguiente:

1. Ve a **/personas/nuevo** y crea 2 personas.
2. Ve a **/personas** y verifica que aparecen.
3. Abre la carpeta **data/** con tu explorador o editor. Deberia haberse creado **personas.json**.
4. Abre ese archivo con tu editor. Deberias ver tus 2 personas formateadas en JSON.
5. Detén el servidor con **Ctrl+C**. Vuélvelo a levantar con **node index.js**.
6. Ve a **/personas**. **Las 2 personas siguen ahi**. Esto antes no pasaba.

Lo logramos

Tus datos ahora persisten. Funcionara mientras no borres la carpeta **data/** ni el archivo. Para una app de prototipo, esto ya es suficiente.

Antes de programar: como funciona la subida de archivos

Cuando enviamos un formulario normal (solo texto), el navegador empaqueta los datos en un formato simple llamado **url-encoded**. Pero cuando hay archivos de por medio, el formato cambia a **multipart/form-data**: el navegador divide el envio en "partes" (multi-part), una por cada campo, y cada parte puede tener su propio tipo (texto, imagen, lo que sea).

Express por defecto no sabe leer este formato. Por eso necesitamos **multer**: un middleware especializado en procesar formularios con archivos.

Que va a hacer multer por nosotros

- Recibir el archivo que viene en el body de la peticion.
- Guardarlo en una carpeta de nuestra eleccion (en este caso **public/uploads/**).
- Renombrarlo para evitar que dos archivos con el mismo nombre se pisen.
- Validar tipo y tamano (si queremos).
- Dejarnos disponible la info del archivo en **req.file**.

Por que guardar en public/uploads

Si guardamos las fotos en **public/uploads/** y ya tenemos **express.static("public")**, las fotos quedan accesibles automaticamente desde la web. Por ejemplo, una foto guardada como **public/uploads/123.png** se puede ver en el navegador en **/uploads/123.png**.

B.1 Instalar multer

En la terminal, dentro de la carpeta de tu proyecto:

```
$ npm install multer
```

Esto descarga la librería y la agrega a las **dependencies** de tu **package.json**.

B.2 Crear la carpeta public/uploads

Dentro de **public/**, crea una carpeta llamada **uploads**. Aquí caerán las fotos que suban los usuarios:

```
public/  
+-- css/  
|   +-- estilos.css  
+-- uploads/           # <- carpeta nueva (vacía por ahora)
```

B.3 Importar multer en index.js

Arriba, junto a los otros **require**:

```
const multer = require("multer"); // nuevo
```

B.4 Configurar como y donde guardar los archivos

Después de los **middlewares**, agrega la configuración de **multer**:

```
// Configurar donde y como guardar los archivos  
const storage = multer.diskStorage({  
  destination: (req, file, cb) => {  
    cb(null, "public/uploads");  
  },  
  filename: (req, file, cb) => {  
    // Nombre único: timestamp + número al azar + extensión original  
    const extension = path.extname(file.originalname);  
    const nombreUnico = Date.now() + "-" + Math.round(Math.random() * 1e6)  
    + extension;  
    cb(null, nombreUnico);  
  }  
});  
  
const upload = multer({ storage: storage });
```

Por qué renombrar los archivos

Si dos personas suben una foto llamada **perfil.jpg**, la segunda sobrescribiría la primera. Renombrando con **Date.now()** (la fecha actual en milisegundos) + un número al azar, garantizamos que cada archivo tenga un nombre único.

B.5

Agregar el campo de archivo al formulario

Abre tu vista **nuevo.hbs** y haz dos cambios en el **<form>**:

1. Agregar el atributo **enctype="multipart/form-data"**.
2. Agregar un **<input type="file">**.

```
<form method="POST" action="/personas"
      enctype="multipart/form-data"  <-- importante
      class="formulario">

  <label>
    Nombres
    <input type="text" name="nombres" required>
  </label>
  <!-- ...el resto de los campos igual... -->

  <label>
    Foto de perfil (opcional)
    <input type="file" name="foto" accept="image/*">
  </label>

  <button type="submit" class="boton">Guardar</button>
</form>
```

El error mas comun aqui

Si te olvidas del **enctype="multipart/form-data"**, el archivo NO llega al servidor. **req.file** sera **undefined** y vas a pensar que multer esta roto. Este atributo es la cosa #1 que se olvida la gente.

Que hace accept="image/*"

Le dice al navegador que filtre los archivos a solo imagenes al momento de elegir. Es una mejora visual, no es seguridad real (el usuario podria saltarsela). La validacion real la haremos despues con multer.

B.6

Usar multer como middleware en la ruta POST

Aqui es donde todo se conecta. Multer va entre `app.post("/personas",` y la funcion `(req, res) => { ... }`. Procesa el formulario, sube el archivo, y nos deja la info lista para usar.

```
app.post("/personas", upload.single("foto"), (req, res) => {  
  //      ^^^ middleware de multer  
  
  const { nombres, apellidos, edad, direccion, password } = req.body;  
  
  // req.file existe SOLO si el usuario subio un archivo  
  const foto = req.file ? "/uploads/" + req.file.filename : null;  
  
  personas.push({  
    nombres, apellidos, edad, direccion, password,  
    foto: foto      // guardamos la RUTA, no el archivo  
  });  
  
  guardarPersonas(personas);  
  res.redirect("/personas");  
});
```

Tres puntos clave

- **upload.single("foto")**: el **"foto"** debe coincidir con el atributo **name** del input de archivo en tu formulario. Si en el HTML tienes **name="avatar"**, aqui debes poner **upload.single("avatar")**.
- **req.file**: contiene info del archivo subido (nombre, tamaño, tipo, ruta). Es **undefined** si el usuario no subio nada.
- En el objeto persona guardamos solo la **ruta publica** de la foto (algo como **/uploads/123.png**), no el archivo completo. El archivo ya vive en el disco; en JSON solo necesitamos saber donde encontrarlo.

B.7

Mostrar las fotos en la lista de personas

Edita tu vista **personas.hbs**. Dentro del **{{#each personas}}**, agrega un **** si la persona tiene foto:

```
{{#each personas}}
  <li class="item">
    {{#if foto}}
      
    {{else}}
      <div class="avatar sin-foto">Sin foto</div>
    {{/if}}

    <h3>{{nombres}} {{apellidos}}</h3>
    <p>Edad: {{edad}}</p>
    <p>Direccion: {{direccion}}</p>
  </li>
{{/each}}
```

Y agregale unos estilos a tu CSS (opcional pero queda lindo):

```
.avatar {
  width: 80px;
  height: 80px;
  border-radius: 50%;          /* redondo */
  object-fit: cover;          /* recortar sin deformar */
}

.sin-foto {
  background: #f0f0f0;
  color: #aaa;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 0.8rem;
}
```

Reinicia el servidor y haz lo siguiente:

1. Ve a **/personas/nuevo**. Veras un campo nuevo para elegir archivo.
2. Llena los datos y elige una imagen cualquiera de tu compu.
3. Presiona Guardar. Te redirige a la lista.
4. La persona aparece con la foto a la izquierda.
5. Abre la carpeta **public/uploads/**: ahi esta el archivo, renombrado.
6. Abre **data/personas.json**: la propiedad **foto** contiene la ruta publica.

Mejora opcional: validar tipo y tamano

Hasta aqui multer acepta cualquier archivo. Para que solo acepte imagenes y limite el peso, expande la configuracion:

```
// Solo permitir imagenes
const filtroImagenes = (req, file, cb) => {
  if (file.mimetype.startsWith("image/")) {
    cb(null, true);
  } else {
    cb(new Error("Solo se permiten imagenes"), false);
  }
};

const upload = multer({
  storage: storage,
  fileFilter: filtroImagenes,
  limits: { fileSize: 2 * 1024 * 1024 } // 2 MB
});
```

Que hace cada cosa

- **fileFilter**: una funcion que decide aceptar o rechazar el archivo. Aqui solo aceptamos los que tienen mimetype empezando con **"image/"** (image/png, image/jpeg, etc).
- **limits.fileSize**: tamaño máximo en bytes. **2 * 1024 * 1024** son 2 megabytes. Si el archivo es mas grande, multer lo rechaza.

Errores comunes y como resolverlos

- **req.file es undefined**

Casi siempre es porque olvidaste **enctype="multipart/form-data"** en el form. Otra causa: el **name** del input no coincide con lo que pones en **upload.single(...)**.

- **La foto no aparece en la lista**

Verifica que la ruta guardada en **foto** sea algo como **/uploads/archivo.png** (con la barra al inicio). Sin la barra el navegador la interpreta como ruta relativa y rompe.

- **Error: ENOENT - no such file or directory**

La carpeta **data/** o **public/uploads/** no existe. Creala antes de iniciar el servidor.

- **Las personas no persisten**

Revisa que la funcion **guardarPersonas(personas)** este DESPUES del **personas.push(...)**. Es facil escribirla antes por error.

- **Error: SyntaxError: Unexpected token en JSON**

Alguien edito **personas.json** a mano y rompio la sintaxis. Borra el archivo y vuelve a crear personas, o arregla el JSON con un validador online.

- **La foto se sube pero el archivo es muy grande**

Agrega **limits.fileSize** a la configuracion de multer. Es buena practica siempre tener un limite.

Retos para practicar

- **Eliminar persona Y su foto**

Crea una ruta para eliminar. Cuando borres una persona, borra tambien su foto del disco con **fs.unlinkSync(...)**. Cuidado: la ruta guardada empieza con **/uploads/...** pero **fs** necesita la ruta del sistema, no la URL.

- **Mostrar fecha de creacion**

Cuando crees la persona, agrega **creadoEn: new Date().toISOString()** al objeto. Mostrala en la lista.

- **Cantidad de personas en el inicio**

Pasa **personas.length** a la vista de inicio y muestra cuantas personas hay registradas.

- **Multiples fotos**

Cambia **upload.single("foto")** por **upload.array("fotos", 5)** para permitir hasta 5 fotos por persona. **req.files** sera un arreglo. Tendras que cambiar el form (**multiple** en el input) y la vista.

- **Avatar por defecto**

Si la persona no tiene foto, en vez de mostrar "Sin foto" muestra una imagen generica que tengas en **public/img/avatar-default.png**.

Lo que aprendimos hoy

Aprendiste a leer y escribir archivos con **fs**, a serializar objetos con **JSON.stringify**, a recibir archivos del cliente con **multer**, y a servir los archivos subidos como contenido estatico.

Lo de hoy te alcanza para cualquier prototipo. Cuando los datos crezcan o necesites busquedas rapidas, llegara el momento de cambiar el JSON por una base de datos. Pero la idea es la misma: cargar al iniciar, guardar al cambiar.